Hybrid reinforcement learning

Yash Palan

July 8, 2026

1 Introduction

The primary objective of the project is the evaluation of quantum-classical neural networks in reinforcement learning (RL). The main objective of this work is to analyze whether the quantum layer contributes beyond what an equivalent classical layer does. The task that the RL agent needs to solve is that of balancing a pole on a cart, namely CartPole-v1 control task.

There are three aspects to the task, namely,

1. Library creation
2. Research question
3. Fault tolerant analysis

Given the 24 hour time budget of the project, my main focus has been on Part b of the question. Below, I summarize how I distributed my effort across the three parts of the project

1. **Library creation**
   - **OOPs based programming:** OOPs concepts were kept in mind while programming the codes, namely modularity was heavily focused on. Inheritance was not as useful for this library.
   - **Config driven experiments:** The hyperparameters can be passed via a "config.json" document or through a similar dictionary in the "train.py" file, which would then take care of training the network.
   - **Docstrings and type hints:**Appropriate type hints and docstrings are added in functions for simplicity of understanding and clarity of the function's intended purpose.
   - **Extensible Design**: The config dependent design also supports both a classical neural net (NN) and hybrid NN.
   - **Error handling**: Basic error handling has been taken into account. However, all possible errors have not necessarily been taken into consideration due to time constraints.
2. **Research question**:
   - **Architecture search:** Taking inspiration from classical neural nets, a quantum layer was defined based on the number of qubits [1]. A schematic of the same is shown in figure **??**. A search was conducted on the number of qubits and the number of quantum layers to find the best architecture.
   - **Quantitative measure of usefulness**: Two quantitative measure of usefulness were defined for the quantum layer. The metrics for the best architecture were then extracted and inferences made based on that.
   - **Comparison with classical baseline:** A classical baseline with similar parameter count was created to compare the quantum results to.

Package name	Version
numpy	2.1.3
scipy	1.17.1
torch	2.11.0+cu126
scikit-learn	1.8.0
gymnasium	1.3.0
pennylane	0.45.1
matplotlib	3.10.8
os	
time	
json	

Table 1: Package version table

- **Reproducibility:** The results for the training of the RL-agent was repeated for 5 unique seeds. The number 5 was chosen simply because of the time constraints. This was done for each architecture. The corresponding plots are shown later in the report.

3. **Fault tolerant computing**:

   - **T gate counting:** The number of T gates for the best performing ansatz were considered and the T gate count and T gate depth were repoted for various ϵ.

2 Library Design

A brief summary of the library design was provided in the previous section. Here we provide more details regarding the abstraction made and provide reasoning for the same.
The main abstaraction that has been made is that of RL agent, where the RL agent is broken into three parts

1. RL agent brain
2. RL agent memory (defined by buffer)
3. RL agent body (everything else that is necessary for training).

The brain is then made up of three neural nets, namely the 2 classical NNs, and one quantum NN sandwitched between the two, as shown in figure 1. The same are defined in different python files. All of this is put into a folder named "`class_defns`". A more detailed version of what each file contains is as below

1. `classical_head_classes.py`**:** Just defines the classical deep neural network class called "`neural_net`".
2. `quantum_circuit_classes.py`**:** Defines the functions that make up the quantum circuit. It constraints
   - Function defining quantum neural net layer
   - Quantum function for making the complete quantum neural net.
   - class which stores the weights and circuit information, as well as the quantum function for making the circuit (similar to the classical case). Note that this class can accept any quantum function for its argument, not just the one defined in this file.
3. `RL_classes.py`**:** This has two classes
   - agent_brain: This class defines the brain of the RL-agent. It has three parts, namely two classical NNs and one quantum NN, as shown in figure 1.

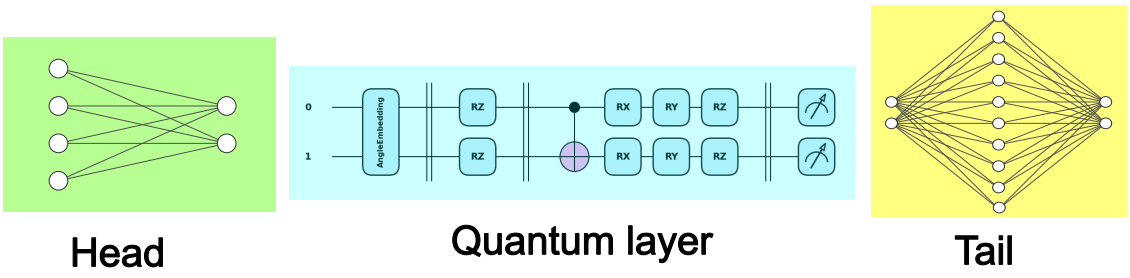

Figure 1: Architecture of the agent's brain

- DQN_agent: This class defines the actual agent and stores all the actions that the agent can take. It can also accepts various inputs for the agent's brain, not just the network defined by the previous class. The only catch is that the agent_brain network must have a forward function (as is in classical NNs).
- Buffer_class: This defines the memory of the RL-agent.

4. `utility_functions.py`: Stores basic utility functions like loading a "`.json`" file or plotting the results of a training run. Can hold more utility functions.

Next, the training and the testing files are kept in the main folder for simplicity purposes. Finally, the remianing files contain the usefulness metric, and the T gate depth and count counting functions. The t gate counting functions have not been abstracted since there was no time left for the same.
Finally, the versions of the packages used in all of the codes is shown in table 1.

3 Experimental setup

The task that we wish to accomplish is balancing a pole on a cart, where the pole has certain initial velocity. This task is generally solved using reinforcement learning (RL) based methods, where an agent learns what actions it needs to perform to maximize the reward (in this case keeping the pole up). There are multiple different strategies that people generally use to solve such problems, however, here we will use the strategy know asn **Deep-Q networks** (DQN). The idea behind a DQN is that instead of learning what move to make next, the agent learns to predict the cumulative future reward that it would get if it makes a certain move. The function which gives the agent this information is called the **Q-function**. The task that we now have to achieve is to find out what the Q function is, which is where we use machine learning based methods. So, the hybrid neural net will be used to approximate the true Q function.
The architecture that we for hybrid quantum-classical neural network is shown in figure 1. The architecture basically consists of a classical NN head, a quantum NN body, and a classical NN tail. This architecture was chosen for the following reasons

1. **Generalizability to mulitple input quantum bits:** This method of encoding allow one to generalize to a variable number of qubits. As an example, if we just had the quantum layer without the head, then the input number of qubits (assuming angle encoding) cannot be more than 4 (for our case since the state vector is 4D). This choice of architecture removes such a restriction.

2. **Input range restriction:** For the angle embedding, we need to restrict our inputs to $[-\pi, \pi)$ or equivalently $(-1, 1)$. However, there can be inputs whose range is $(-\infty, \infty)$. So, it is essential for us to find a clip for the same. The head basically learns this clip.

3. **Encoding feasibility for the quantum layer:** The clipping nature of the head also allows for the encoding to be more dense, since it allows us to use angle encoding. Otherwise, we would need to encode each bit in a qubit, making the encoding space too large.

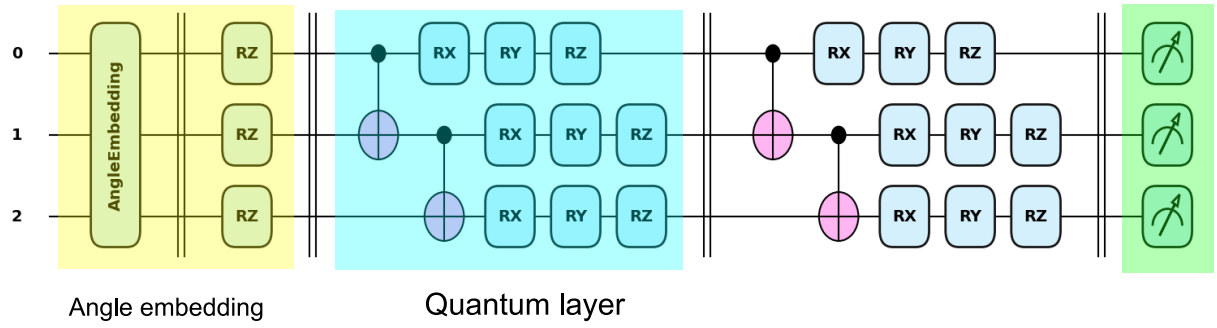

Figure 2: Architecture of the quantum layer. The yellow box represents the angle embedding part, the blue box the individual repeating quantum layer and the green the σ_Z measurement. This architecture has been adapted from [1]

.

4. **Interpretation of quantum layer:** Finally, the tail also allows one to effective sum over the final outputs of the quantum layer. It basically acts as an interpreter/ decoder of the quantum layer.

With regards to the actual quantum NN, it is made of mainly two components,

1. **Input encoding:** The input encoding is essential to be able to encode the input vectors into a form that needs less qubits. I have used angle based encoding, with a combination of $R_x(\phi), R_z(\phi)$ gates for each value in the input vector. The same is shown in figure 2 as the green box.
2. **Quantum layer:** The quantum layer is the layer which can be repeated to create the whole circuit. An example of the same is shown in figure 2.

Thus our hyperparameters are

1. **Head:**
   - Layer geometry of the head
2. **Quantum circuit:**
   - Number of qubits
   - Number of layers
3. **Tail:**
   - Layer geometry of the tail.

By varying the number of qubits and the number of layers, we can obtain different ansätz for the quantum layer/ quantum NN.
Remark - Details regarding the seeds and baselines will be given in the results sections

4 Usefulness metrics

As is defined in the project, the task is not to find a hybrid model that's better but to be able to evaluate whether the quantum layer truly adds to the working of the neural network and if so how. We thus need quantitatives measures which could describe the quantum layer and also what its effects.
One direction to be motivated by is the fact that quantum states are useful because of the property of

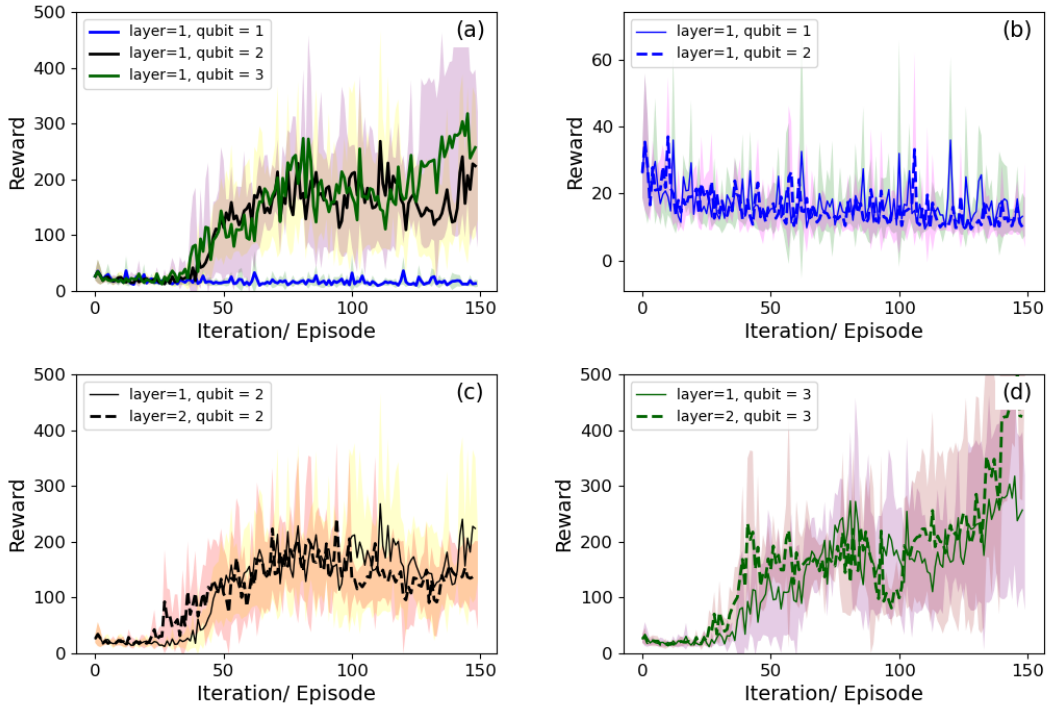

Figure 3: Comparison of the various ansätz for the quantum layer. Note that the shaded regions denote the error bars of the system. The error bars are taken to be one sigma above and below the mean.

superposition and entanglement. Thus, if could extract this information, then that would tell us about the working of the quantum layer. However, this is only useful when restricted to inputs for the trained enviornment and thus we only consider input vectors which are a states of the enviornment. Keeping the above motivation in mind, we can define the following two quantities

1. **Compression ratio:** The compression ratio computes the amount of compression that the quantum layer performs. So, given the input of the quantum layer as $\vec{\text{in}}$ and the output be $\vec{\text{out}}$, one can consider the dimension of the subspace that encodes 95% of the information for both the input vectors and output vectors. Let the dimensions be d_{in} and d_{out}. Then, we can define the **Compression ratio** C as

$$C = 1 - \frac{d_{out}}{d_{in}}. \tag{1}$$

   The gives us information about how much of the information could be compressed by the qunatum layer.

2. **Average entanglement entropy:** For systems with multiple qubits, entanglement entropy is not defined normally. So, we compute the bipartite entropy

$$\mathcal{S}(\rho_i) = -\operatorname{Tr}[\rho_i \log(\rho_i)], \tag{2}$$

   where ρ_i is the reduced density matrix for qubit i. Thus, given input vectors, $\vec{\text{in}}$, we can look at the output vectors $\vec{\text{in}}$ and compute the entanglement entropy corersponding to the output state and qubit

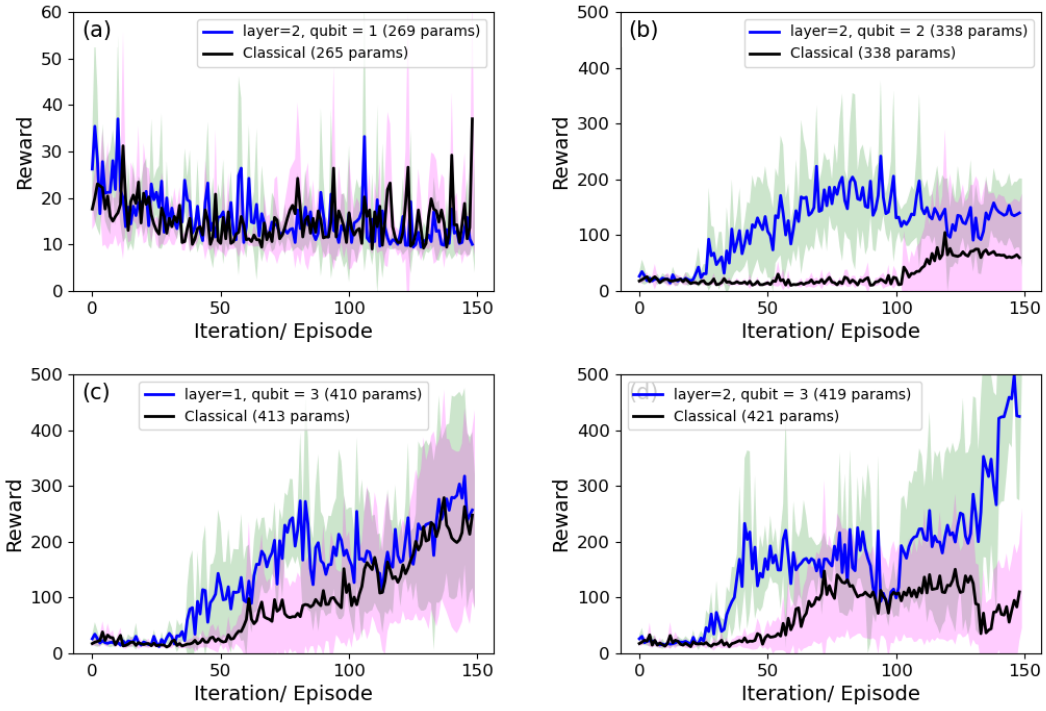

Figure 4: Comparison of the quantum model with the classical benchmarks. Note that the shaded regions denote the error bars of the system. The error bars are taken to be one sigma above and below the mean.

.

i $\left(\mathcal{S}(\rho_i^{\vec{\text{out}}})\right)$. The averaged entanglement entropy is thus

$$\mathcal{A}_i = \sum_{\vec{\text{out}}} \mathcal{S}(\rho_i^{\vec{\text{out}}}). \tag{3}$$

The above provides us information about

- Freedom of measurement: Since the output state is supposed to be based on measurements, this gives us information about how correlated measurements would be.
- Link to Barren plateau problem: This can also provide some insight into the barren plateau problem (since we can look at the probability distribution and compare it to the Haar distribution).

5 Results

Having described the architecture of the system and the metrics for understanding the quantum layer, we can now start asking the true question, which quantum layer architecture is the best for the RL CartPole-v1 problem and how it fare compared to the classical benchmarks?
To do so, a small search over the qubit number $(1, 2, 3$ qubits) and layers $(1, 2$ layers) was performed, giving us a total of 6 architectures. Each architecture was trained 5 times with a unique seed for each training run. The seeds were chosen to be $0, 1, 2, 3, 4$ for simplicity. Finally, the results obtained from each training run

Parameter count	Architecture	Quantum equivalent
265	$[4, 1, 1, 64, 2]$	Qubit = 1
338	$[4, 2, 2, 64, 2]$	Qubit = 2
413	$[4, 3, 3, 64, 2]$	Qubit = 3, layer = 1
421	$[4, 4, 3, 64, 2]$	Qubit = 3, layer = 2

Table 2: This table depcits the architecture of the classical NN used for the baseline, as well as the quantum version it is compared to.

Layer	Qubits		qubit 1	qubit 2	qubit 3
2	1	mean	$-2.6229 \times 10^{-8} \approx 0$	NA	NA
		standard deviation	$2.1312 \times 10^{-15} \approx 0$	NA	NA
2	2	mean	0.3466	0.3466	NA
		standard deviation	0.0190	0.0190	NA
2	3	mean	0.3305	0.7390	0.6517
		standard deviation	0.0666	0.0137	0.0128

Table 3: Here we show the average entanglement entropy for the different architectures. Note that the entanglement entropy does not grow for the first qubit, but does for the second and the third.

(the reward as a function of episode) were averaged and the standard deviation of the same was found, for a each architecture. This was then used to make the plots figure 3.
Finally, we also needed a classical benchmark for each of these architectures. Thus, the parameter count for each architecture was extracted and then an equivalent classical NN was created for the same. The architecture for the same is shown in table 2.
We can also ask the results for our metrics, namely compression ratio and average entanglement entropy. The results for the same are

1. **Compression ratio:**
   - The compression ration for the best quantum circuit was 0, indicating that no compression took place.
   - In this sense, this metric falls short since
2. **Average entanglement entropy:** The results for the average entanglement entropy is shown in table 3. It is important to notice that average entanglement in the second qubits for the qubit 3 case is higher then the qubit 2 case.

6 Discussion

Having briefly presneted the results for the metrics, we now turn to the inferences that we can make from the everything that has been presented the previous sections.

1. **Best quantum architecture:** The best quantum architecture is the case with **layer** $= 2$**, qubits** $= 3$. However, one can easily argue that **layer** $= 1$**, qubits** $= 2$ is an equivalently good contender, and indeed, this result is not statistically significant (upto one sigma), however, based on just the mean, **layer** $= 2$**, qubits** $= 3$ is the best.
2. **Classical vs hybrid:** In terms of the final trained RL agent, the hybrid version does not outshine the classical network. This is clearly seen from figure 4 (a),(b) and (c), where the classical network catches up to the quantum network.

Layer	Qubits	T gate count		Depth	
		$\epsilon = 1e-1$	$\epsilon = 1e-3$	$\epsilon = 1e-1$	$\epsilon = 1e-3$
1	3	360	160	152	330
2	3	410	782	426	778

Table 4: Here we show the T count and depth for the case when we consider two ansätz with 3 qubits each. The ϵ is the precision to which the gates are approximated.

3. **Effect of entanglement on the system:** It is interesting to note that the increase to the optimal if faster for the quantum version than the classical, specifically in the 2 and 3 quibit cases (notice the strak rise at $iteration = 50$ for the hybrid vs the classical case). From our metric, we notice that the entanglement here is higher, indicating that maybe larger entanglement plays a role in convergence to the optimal solution.

4. **Entanglement vs superposition:** Now, one may also ask why entanglement and not superposition? The answer to the same is as follows: assume that it was indeed superposition due to which we observe better convergence properties, then it should also be seen in the 1 qubit hybrid model. However, the 1 qubit hybrid model matches the classical model exactly, indicating superposition cannot be the reason.

Conclusion 1: The hybrid network has faster convergence properties (upto one sigma significance) than the classical networks. From an accuracy perspective, neither one is better than the other.
Conlusion 2: Quantum layer working: Thus the quantum layer speeds up convergence via the mechanism of entanglement, which was not existent in the classical case.

7 Fault tolerance probe

The final analysis that is necessary is the understanding of the real world applicability of the above circuits. However, the only gates that can be made in the real world are the Clifford+T gates, with the T gates introducing error, and the depth introducing error via decoherence. Thus, an understanding of the T gate count and depth provides us with insight on how fault tolerant the hybrid system wiil be. Keeping this in mind, I evaluated the T gate count for the layer $1, 2$ for the 3 qubit case. The same is shown in table 4.
Clearly, more layers leads to larger depth and more T gates. A larger depth and larger number of T gates basically implies that the final results of such a circuit in real life will not be reliable due to accumulation of errors from decoherence and T gate implementation. Thus, reducing the T count and depth becomes extremely necessary to make hybrid quantum circuits that are truly fault tolerant.

8 Limitations and next steps

8.1 Limitations

Having discussed the conclusion and fault tolerant computing (briefly), it is important to point out the limitations of this study. The limitation are as follows

1. **Statistics:** A major limitation in this study is the number of seeds used for producing the plots. Claerly, better statistics would provide more robust results and give better variance bounds, making it easier to make conclusion which are statistically more significant. This is necesasy to make concrete statements, especially of the kind made here.

2. **Time complexity:** Part of the reason for low statistics is the time complexity for computing a single episode for an architecture with 3 qubits and 2 layers. Thus, time complexity of the code needs to be made better to allow for a larger varierty.

3. **Head architecture variation:** In all of the simulations, we have not actually seen the effect of the variation of the head and the tail NNs on the hybrid case. This would be interesting since it would allow us information about how much **abstract encoding can the quantum layer work with**[**<empty citation>**].

8.2 Next steps

Due to the time constraints, a lot of stuff that should be inverstigated could not be. Here I will mention a few of the future directions where this work could be extremely useful. I break this in 3 parts

1. **Immediate future**
   - Make the time complexity of the code better to allow for larger statistics.
   - Take into account larger number of training runs to get better statistics and make the conclusions even stronger.
2. **A little longer future**
   - The next step would be to also make a search on the head architecture and the tail architecture fixing the quantum layer to see the effect of the same on the hybrid system.
   - A secondary extension could be to now use autoencoder based techniques and see how a quantum layer maybe useful in that case.
   - Another possibility would be to use another RL based method, like Policy optimisation, and check if the hybrid methods work well there.
3. **Speculative future**
   - Having noticed that convergence is where quantum layers work best, one can try to consider another task, like say playing tetris and see if there we observe a similar behaviour when compared to the classical case.
   - The RL based technique can also be used for circuit creation on its own using policy based methods. So, then one can consider using the same to create dynamical circuits that approximate physical Hamiltonian. This type of work would be interesting as that can then help speed up research work since finding relevant Hamiltonians is the hard part. In addition, adding data driven search would make this even better.

9 LLM usage disclosure

Naturally, free tier LLMs were used during the work. LLMs were used in the following ways

1. **Literature search:** The free tier LLMs were used to search out papers on hybrid quantum techniques in RL as well as literature on various ansätz and T gates.
2. **Learning basics:** In addition, a small time was spent searching and understanding some basics necessary to working. As an example, simple questions about the working of pennylane, and of DQNs were asked (textbook level question), which were verified by citations given there or by looking it up on google as well.
3. **Generation of small code snippets:** During the coding process, LLMs were used to generate small snippets of code and check up on syntax of commands that I did not know well (in addition to documentation).
4. **Soundboard for idea consolidation:** Also, LLMs provide a good soundbaord for ideas of what I wished to do and what I would like to do. It was a good place to just write my thoughts and get some thing back (which was not always correct or even related).

10 References used

10.1 For coding

1. *ArtificialIntelligence/DQN/dqn.ipynb at main · JavierMtz5/ArtificialIntelligence — github.com*. https://github.com/{J}avier{M}tz5/{A}rtificial{I}ntelligence/blob/main/{D}{Q}{N}/dqn.ipynb. [Accessed 08-07-2026] was used as a reference to write the basic DQN training loop and code for the classical case. This was then extended to the quantum case.

2. *Inspecting circuits — PennyLane — docs.pennylane.ai*. https://docs.pennylane.ai/en/stable/introduction/inspecting_circuits.html. [Accessed 08-07-2026]: A short code snippet for the output of the T gate characterisation was used from here.

10.2 For literature survey

1. Owen Lockwood and Mei Si. *Reinforcement Learning with Quantum Variational Circuits*. 2020. arXiv: 2008.07524 [quant-ph]. URL: https://arxiv.org/abs/2008.07524: The main ansätz for the individual layer was taken from here.

2. Silvie Illésová, Tomasz Rybotycki, and Martin Beseda. *QMetric: Benchmarking Quantum Neural Networks Across Circuits, Features, and Training Dimensions*. 2025. arXiv: 2506.23765 [quant-ph]. URL: https://arxiv.org/abs/2506.23765: This was used to motivate the first measure of quantum layer usefullness

3. Amira Abbas et al. "The power of quantum neural networks". In: *Nature Computational Science* 1.6 (2021), pp. 403–409. ISSN: 2662-8457. DOI: 10.1038/s43588-021-00084-1. URL: http://dx.doi.org/10.1038/s43588-021-00084-1: For understanding effective dimension (briefly)

4. Dániel T. R. Nagy et al. "Hybrid quantum-classical reinforcement learning in latent observation spaces". In: *Quantum Machine Intelligence* 7.2 (2025). ISSN: 2524-4914. DOI: 10.1007/s42484-025-00306-z. URL: http://dx.doi.org/10.1007/s42484-025-00306-z

5. Jarrod R. McClean et al. "Barren plateaus in quantum neural network training landscapes". In: *Nature Communications* 9.1 (Nov. 2018). ISSN: 2041-1723. DOI: 10.1038/s41467-018-07090-4. URL: http://dx.doi.org/10.1038/s41467-018-07090-4

6. Peter Henderson et al. *Deep Reinforcement Learning that Matters*. 2019. arXiv: 1709.06560 [cs.LG]. URL: https://arxiv.org/abs/1709.06560

7. Kamila Zaman et al. *A Comparative Analysis of Hybrid-Quantum Classical Neural Networks*. 2024. arXiv: 2402.10540 [quant-ph]. URL: https://arxiv.org/abs/2402.10540

8. Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG]. URL: https://arxiv.org/abs/1312.5602: For understanding the DQN idea and what it does, as well as the Bellmann equation.

9. Samuel Yen-Chi Chen et al. *Variational Quantum Circuits for Deep Reinforcement Learning*. 2020. arXiv: 1907.00397 [cs.LG]. URL: https://arxiv.org/abs/1907.00397

10. Aueaphum Aueawatthanaphisut and Nyi Wunna Tun. *Hybrid Quantum-Classical Policy Gradient for Adaptive Control of Cyber-Physical Systems: A Comparative Study of VQC vs. MLP*. 2025. arXiv: 2510.06010 [quant-ph]. URL: https://arxiv.org/abs/2510.06010

11. Maureen Monnet et al. *Pooling techniques in hybrid quantum-classical convolutional neural networks*. 2023. arXiv: 2305.05603 [quant-ph]. URL: https://arxiv.org/abs/2305.05603

References

[1] Owen Lockwood and Mei Si. *Reinforcement Learning with Quantum Variational Circuits*. 2020. arXiv: 2008.07524 [quant-ph]. URL: https://arxiv.org/abs/2008.07524.

[2] *ArtificialIntelligence/DQN/dqn.ipynb at main · JavierMtz5/ArtificialIntelligence — github.com*. https://github.com/{J}avier{M}tz5/{A}rtificial{I}ntelligence/blob/main/{D}{Q}{N}/dqn.ipynb. [Accessed 08-07-2026].

[3] *Inspecting circuits — PennyLane — docs.pennylane.ai*. https://docs.pennylane.ai/en/stable/introduction/inspecting_circuits.html. [Accessed 08-07-2026].

[4] Silvie Illésová, Tomasz Rybotycki, and Martin Beseda. *QMetric: Benchmarking Quantum Neural Networks Across Circuits, Features, and Training Dimensions*. 2025. arXiv: 2506.23765 [quant-ph]. URL: https://arxiv.org/abs/2506.23765.

[5] Amira Abbas et al. "The power of quantum neural networks". In: *Nature Computational Science* 1.6 (2021), pp. 403–409. ISSN: 2662-8457. DOI: 10.1038/s43588-021-00084-1. URL: http://dx.doi.org/10.1038/s43588-021-00084-1.

[6] Dániel T. R. Nagy et al. "Hybrid quantum-classical reinforcement learning in latent observation spaces". In: *Quantum Machine Intelligence* 7.2 (2025). ISSN: 2524-4914. DOI: 10.1007/s42484-025-00306-z. URL: http://dx.doi.org/10.1007/s42484-025-00306-z.

[7] Jarrod R. McClean et al. "Barren plateaus in quantum neural network training landscapes". In: *Nature Communications* 9.1 (Nov. 2018). ISSN: 2041-1723. DOI: 10.1038/s41467-018-07090-4. URL: http://dx.doi.org/10.1038/s41467-018-07090-4.

[8] Peter Henderson et al. *Deep Reinforcement Learning that Matters*. 2019. arXiv: 1709.06560 [cs.LG]. URL: https://arxiv.org/abs/1709.06560.

[9] Kamila Zaman et al. *A Comparative Analysis of Hybrid-Quantum Classical Neural Networks*. 2024. arXiv: 2402.10540 [quant-ph]. URL: https://arxiv.org/abs/2402.10540.

[10] Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG]. URL: https://arxiv.org/abs/1312.5602.

[11] Samuel Yen-Chi Chen et al. *Variational Quantum Circuits for Deep Reinforcement Learning*. 2020. arXiv: 1907.00397 [cs.LG]. URL: https://arxiv.org/abs/1907.00397.

[12] Aueaphum Aueawatthanaphisut and Nyi Wunna Tun. *Hybrid Quantum-Classical Policy Gradient for Adaptive Control of Cyber-Physical Systems: A Comparative Study of VQC vs. MLP*. 2025. arXiv: 2510.06010 [quant-ph]. URL: https://arxiv.org/abs/2510.06010.

[13] Maureen Monnet et al. *Pooling techniques in hybrid quantum-classical convolutional neural networks*. 2023. arXiv: 2305.05603 [quant-ph]. URL: https://arxiv.org/abs/2305.05603.